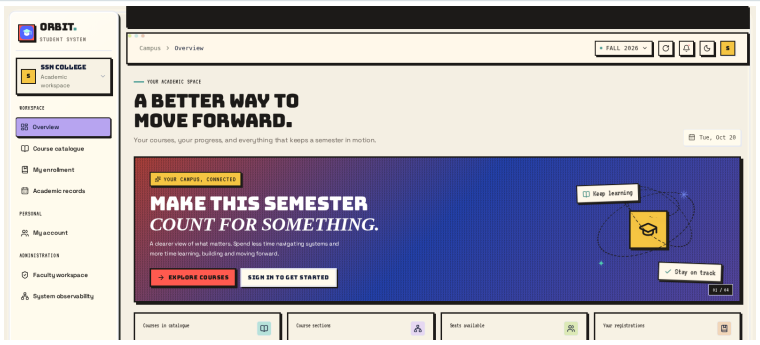
SYSTEM DESIGN LAB

Designing a Student Information System Around Correctness

Architecture, transactional enrollment, and evidence-based verification



Student and staff workflows shown in the local application using synthetic demonstration data.

REPORT DETAILS

Document	Technical system-design report
Review	Design and verification
Date	21 October 2026
Verification environment	Local prototype; synthetic data; PostgreSQL 17.11

PROJECT TEAM

Tanish S K

3122245001180

Tarun S

3122245001182

Sujith M

3122245001172

Velamapalli Hemanth Raj Sreyash

3122245001195

Coursework prototype. Results document tested local behavior; they are not production certification or capacity claims.

Abstract

Student information systems combine high-contention registration workflows with sensitive academic records. This report presents the design and local verification of Orbit SIS, a full-stack prototype built with React, FastAPI, SQLAlchemy, and PostgreSQL. The principal design decision is to make PostgreSQL the authority for enrollment capacity: a conditional seat-reservation update and the enrollment insert execute in one transaction, with rollback on conflict. In an isolated PostgreSQL 17.11 verification environment, the automated suite reported 23 passing tests; two targeted concurrency tests checked seat-capacity and duplicate-enrollment invariants. Fresh-database migrations, schema-drift checks, Ruff, the frontend production build, five live API smoke checks, and seven desktop UI captures also passed in the recorded run. The report separates these results from historical, single-run Locust measurements and from process-local demonstrations of caching, rate limiting, hash-ring routing, and telemetry. On 21 October, the PostgreSQL suite was rerun after changing the 64-bit ID API to return strings; Postman collection verification passed 24 assertions, and a controlled Apache JMeter read workload completed with zero errors. An unpaced JMeter overload run also demonstrated HTTP 429 admission control. These are local checks, not production-capacity claims. The report documents limitations, reproduction steps, and the next engineering gates. The evidence supports the tested local behaviors; it does not establish production security, availability, or scale.

Keywords: student information system, transactional consistency, PostgreSQL, concurrent enrollment, system design, verification

Contents

Abstract i

1 Problem and design objectives 1

1.1 Context . . . 1
1.2 Design objectives . . . 1
1.3 Scope and non-goals . . . 1

2 Architecture and data model 2

2.1 Runtime architecture . . . 2
2.2 Core entities . . . 2
2.3 Consistency boundary . . . 2

3 Transactional enrollment correctness 3

3.1 Invariant . . . 3
3.2 Request sequence . . . 3
3.3 Concurrent capacity test . . . 3
3.4 Duplicate-request race . . . 3

4 Implementation and design trade-offs 5

4.1 Component boundaries . . . 5
4.2 Security and privacy scope . . . 5

5 Verification methodology and results 6

5.1 Environment and isolation . . . 6
5.2 Verification matrix . . . 6
5.3 Interface evidence . . . 6

6 Data integrity and domain model 10

6.1 Separate catalogue identity from a term-specific offering . . . 10
6.2 Integrity rules and enforcement layers . . . 10
6.3 Idempotency is not the same as uniqueness . . . 10

7 Architecture decisions and alternatives 11

7.1 Decision: a modular API with one authoritative relational store . . . 11
7.2 Rejected or deferred alternatives . . . 11
7.3 Why the consistency boundary matters . . . 12
7.4 Operational boundaries of auxiliary components . . . 12

8 Test strategy, reproducibility, and evidence quality **13**
8.1 Layered verification strategy . . . 13
8.2 Isolation and repeatability . . . 13
8.3 What the evidence does and does not establish . . . 13
8.4 Benchmark interpretation . . . 13

9 Security, privacy, and release readiness **15**
9.1 Current safeguards and residual risks . . . 15
9.2 Release gates . . . 15

10 Performance observations **16**
10.1 Postman and Apache JMeter verification — 21 October 2026 . . . 16
10.2 Historical local measurements . . . 20
10.3 Stack verification after the infrastructure migration . . . 20
10.4 Limitations . . . 20

11 Mobile layout quality assurance **22**

12 Conclusion and next steps **23**

References **24**

A Reproduction checklist **25**

B Evidence interpretation **26**

List of Figures

2.1 Runtime architecture 2

3.1 Enrollment transaction path 3
3.2 Expected and observed outcome distribution for the five-seat concurrency test. . 3

5.1 Dashboard overview 7
5.2 Course catalogue 7
5.3 Successful enrollment 8
5.4 Faculty workspace 8
5.5 System observability 9

10.1 Postman collection results 17
10.2 Apache JMeter controlled baseline 18
10.3 Apache JMeter overload behavior 19

11.1 Mobile layout after the responsive offset fix, captured at 390 × 844. This is targeted geometry regression evidence, not a full accessibility audit. 22

List of Tables

2.1 Core domain entities and responsibilities 2

4.1 Implemented mechanisms and their current boundaries 5

5.1 Recorded verification results, 20 October 2026 6

6.1 Integrity concern, enforcement point, and failure prevented 10

7.1 Architecture alternatives and reasons for deferral 11

8.1 Evidence strength and limits 14

9.1 Minimum next gates before a non-local deployment 15

10.1 Tool-assisted API verification, 21 October 2026 16

10.2 Historical single-run Locust observations (19 October 2026) 20

10.3 Measured behaviour against Redis 7 and a two-shard MongoDB 7 cluster (21 October 2026) 21

Chapter 1

Problem and design objectives

1.1 Context

Registration is a consistency-sensitive workflow. Several students may request the last available seat at nearly the same time. A design that checks capacity in the application and updates it later leaves a race window: two requests may observe the same remaining seat. Academic records also require controlled access, predictable validation, and a clear authoritative data store.

Orbit SIS was developed as a local full-stack prototype for student and staff workflows, with additional system-design experiments that demonstrate caching, rate limiting, ID generation, short links, crawling, routing, and telemetry. The report focuses on what was implemented and what was verified, rather than presenting each experiment as production infrastructure.

1.2 Design objectives

1. Preserve enrollment invariants under concurrent requests.
2. Keep authentication, authorization, and request validation in the API layer.
3. Keep persistent academic state in PostgreSQL.
4. Make component boundaries and trade-offs explicit.
5. Support reproducible local checks and distinguish fresh results from historical measurements.

1.3 Scope and non-goals

The verification covers a local prototype and synthetic records. It does not certify production security, high availability, disaster recovery, multi-region behavior, or production throughput. Rate limiting, caching and sharded records are backed by real services (Redis 7 and a two-shard MongoDB cluster), but each of those services is a single node on one host: their state is shared between API workers, their availability is not. The crawler was verified against the live public network and honours robots.txt and `Crawl-delay`.

Chapter 2

Architecture and data model

2.1 Runtime architecture

The React interface presents student and faculty workflows. FastAPI exposes HTTP endpoints and applies validation and access checks. SQLAlchemy connects the API to PostgreSQL, which stores student profiles, courses, offerings, enrollments, and academic records. The database is the source of truth for seat capacity and enrollment state. The transaction and locking model follows PostgreSQL's documented concurrency mechanisms [3, 4].

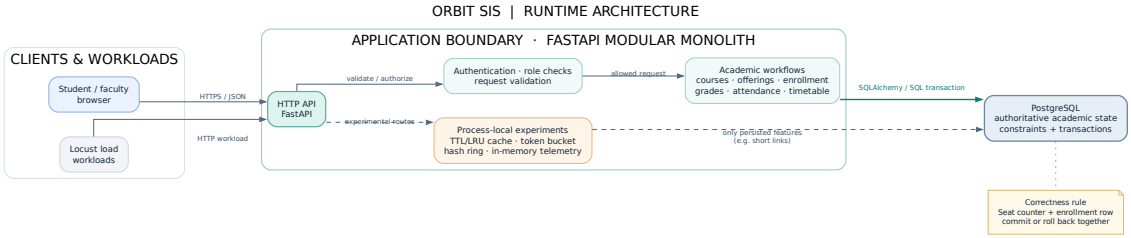


Figure 2.1. Runtime architecture and trust boundaries. Solid edges show primary request/data paths; dashed edges indicate experimental or auxiliary paths.

2.2 Core entities

Table 2.1. Core domain entities and responsibilities

Entity	**Responsibility**
midrule Student	Identity, profile, and student-specific access scope.
Course	Catalogue metadata shared across offerings.
Course offering	Term/section, status, capacity, and enrolled count.
Enrollment	Student-to-offering relationship, state, and request/idempotency context.
Academic records	Attendance, assessment, grades, timetable, and notifications.
bottomrule	

2.3 Consistency boundary

The cache and the rate limiter are backed by Redis, and their state is coordinated across API workers: the token bucket's read-modify-write is a single Lua script, and the cache is a server-side `SETEX` entry. Sharded records live behind a `mongos` router that owns chunk placement. None of these is authoritative for academic state. PostgreSQL constraints and transactional updates protect enrollment; a cache hit or routing decision must never determine whether a seat exists.

Chapter 3

Transactional enrollment correctness

3.1 Invariant

For each offering, the primary capacity invariant is

$$0 \leq \texttt{enrolled_count} \leq \texttt{capacity}. \tag{3.1}$$

The endpoint performs a conditional update that reserves a seat only while the offering is open and the enrolled count is below capacity. The enrollment insert occurs in the same transaction. If the insert fails because of a uniqueness conflict, the transaction rolls back the counter update.

3.2 Request sequence

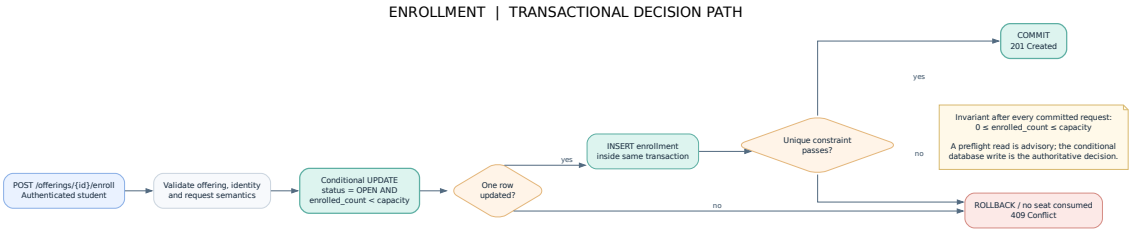


Figure 3.1. Enrollment decision path. A conditional database write decides capacity; the seat update and enrollment insert commit or roll back together.

3.3 Concurrent capacity test

Twenty distinct students concurrently attempted enrollment into an offering with five seats using 20 worker threads. The test expected five HTTP 201 responses, fifteen HTTP 409 responses, an enrolled count of five, and five active enrollment rows. The recorded PostgreSQL-backed run passed this invariant.

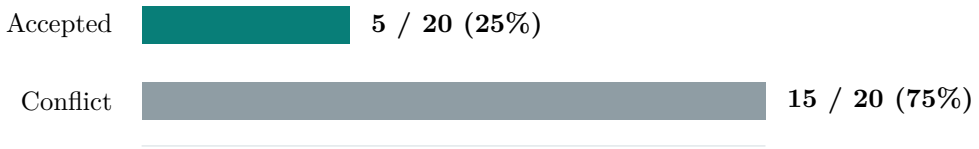


Figure 3.2. Expected and observed outcome distribution for the five-seat concurrency test.

3.4 Duplicate-request race

A second test submitted two simultaneous enrollment attempts for the same student and offering with distinct idempotency keys. The test asserts one success, one conflict, one active enrollment,

and a single counter increment. This is targeted evidence for the tested race, not proof that every endpoint or every deployment condition is race-free.

Chapter 4

Implementation and design trade-offs

4.1 Component boundaries

Table 4.1. Implemented mechanisms and their current boundaries

Component	Demonstrated behavior	Boundary and next step
midrule PostgreSQL	Persistent academic state, conditional seat reservation, uniqueness constraints.	Add backup/restore drills, connection-pool tuning, and operational monitoring.
TTL cache in Redis	Autocomplete reuse shared by every API worker; server-side expiry.	A single unreplicated Redis node with a 128 MB `allkeys-lru` cap; an entry can be evicted before its TTL.
Token-bucket limiter in Redis	Burst control whose counters are shared by every API worker.	One Lua script performs the whole read-modify-write, so workers cannot double-spend; a Redis outage degrades to HTTP 503, never to unlimited traffic.
Consistent hash ring	Virtual-node routing over the live shard set, reported beside the router's own answer.	It routes only; it never moves or replicates data.
Sharded records in MongoDB	`mongos` router, config-server replica set, two shard replica sets; collection sharded on `_id: hashed`.	Two shards on one host with single-member replica sets; real `moveChunk` and rebalance are exposed, but no host-level failover.
Telemetry	In-process request counters plus live Redis and cluster state.	Counters reset on restart; production monitoring needs persistent metrics and dashboards.
Crawler	robots.txt (RFC 9309), `Crawl-delay` pacing, retry on 429/503, durable frontier in MongoDB.	Same-host and serial by design; DNS rebinding is not defended against, so it must not be a public arbitrary-URL fetch service.
ID endpoint	Returns 64-bit identifiers as JSON strings; uniqueness is asserted before serialization.	Clients must treat IDs as opaque strings; a string response avoids JavaScript Number precision loss.
bottomrule		

4.2 Security and privacy scope

The prototype includes role/self-access checks for staff-only academic mutations and student record reads. Staff promotion is a trusted local command-line operation. Token revocation/refresh and production identity controls are not implemented. The application should not be exposed publicly or populated with real student records without additional security, privacy, and operational review. The crawler also requires independent review before use outside coursework.

Chapter 5

Verification methodology and results

5.1 Environment and isolation

The recorded verification used a dedicated PostgreSQL 17.11 container on port 55432 with separate databases for synthetic demo data and the automated test suite. This avoided modifying the pre-existing `sis` database. Three migration revisions were applied to fresh databases. The UI was served from the production frontend build and used synthetic demo records.

5.2 Verification matrix

Table 5.1. Recorded verification results, 20 October 2026

Check	Result	Scope
midrule Automated test suite	23 passed	Full suite against isolated PostgreSQL database.
Capacity race	Passed	20 concurrent students compete for five seats.
Duplicate enrollment race	Passed	One active enrollment and one counter increment after concurrent duplicate requests.
Alembic migrations	Passed	Three revisions applied to fresh databases.
Schema drift check	Passed	No new upgrade operations detected.
Ruff	Passed	Application, tests, scripts, and migrations.
Frontend build	Passed	TypeScript and Vite production build.
Live API smoke checks	5 / 5	Health, OpenAPI, course list, autocomplete, and ID endpoint returned HTTP 200.
Desktop UI capture	7 sections	Overview, catalogue, enrollment, records, account, faculty, and observability.
Browser JavaScript errors	0 observed	During the recorded navigation/capture run only.

The automated suite emitted an upstream Starlette/httpx test-client deprecation warning. It did not fail the suite, but the dependency stack should be reviewed during maintenance. The public crawler was not verified because DNS/network reachability was unavailable.

5.3 Interface evidence

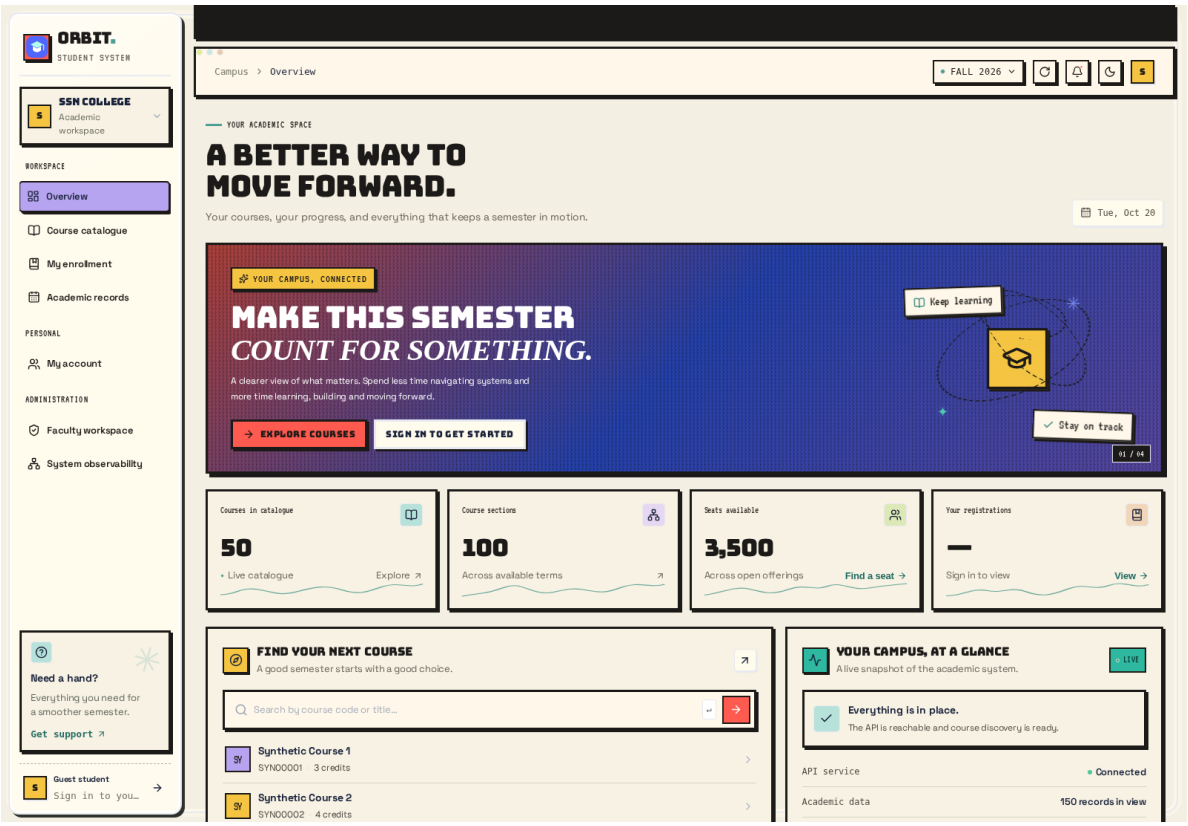


Figure 5.1. Dashboard overview captured from the production frontend with synthetic verification data.

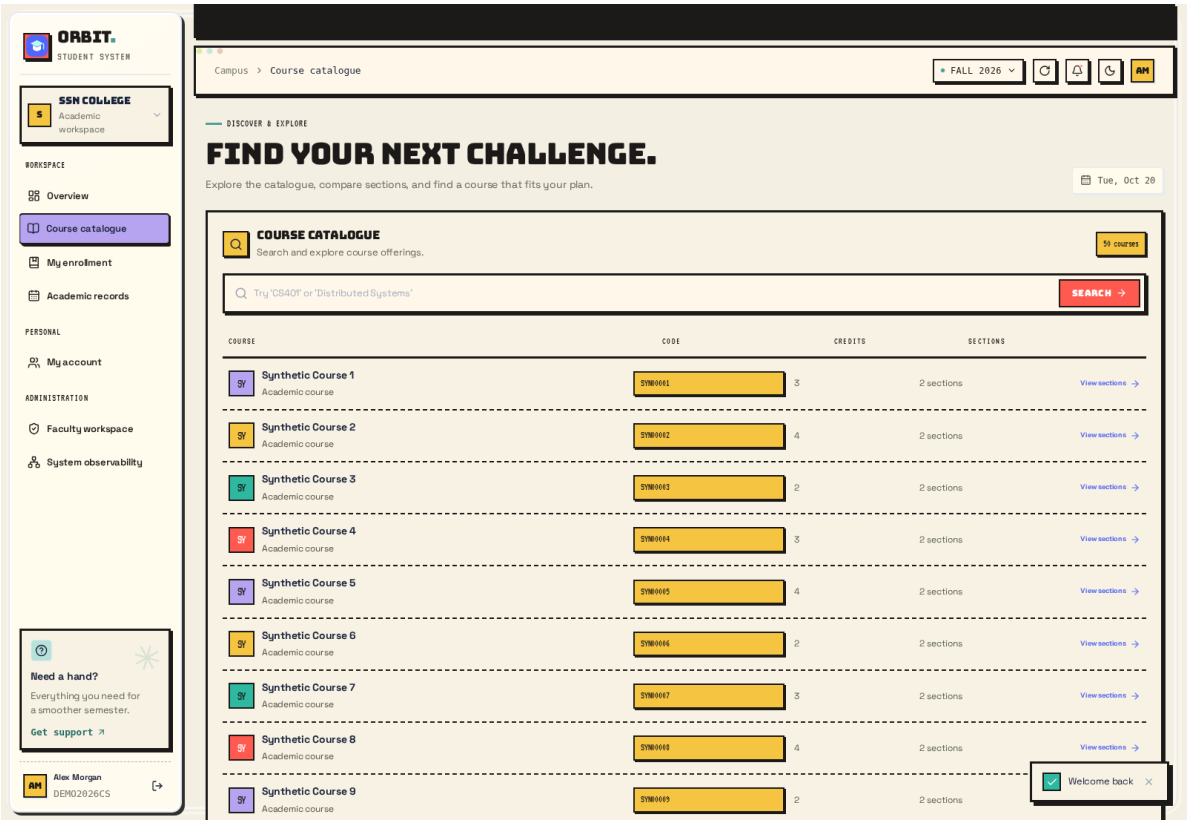


Figure 5.2. Course catalogue and offering discovery workflow.

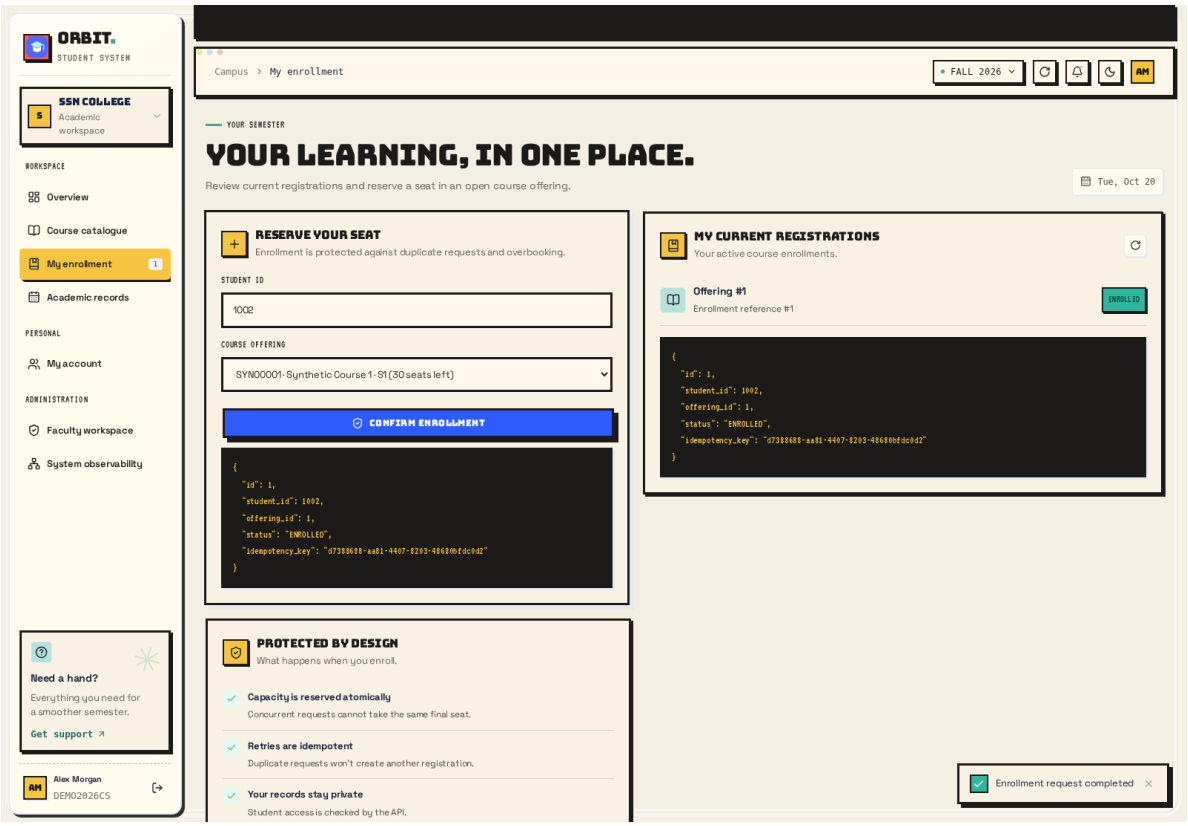


Figure 5.3. Successful synthetic student enrollment. The displayed record is demonstration data.

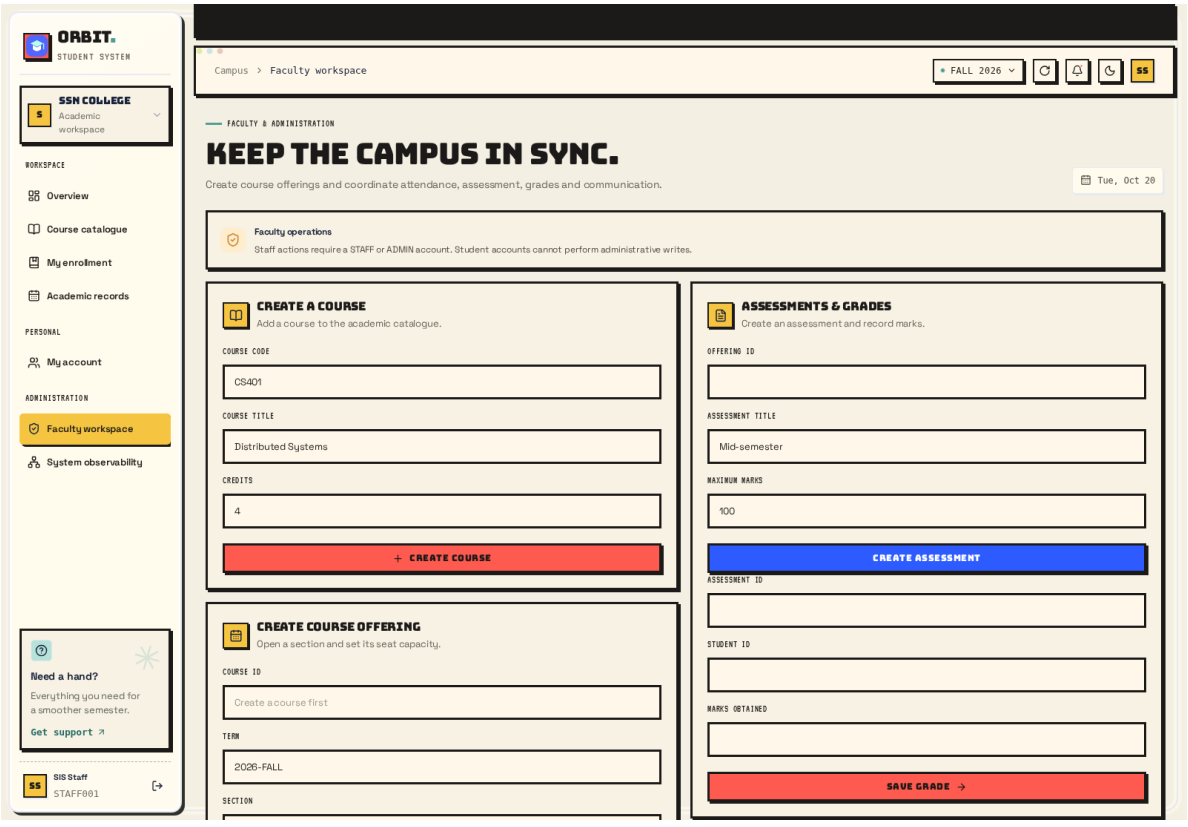


Figure 5.4. Faculty workspace captured after staff sign-in.

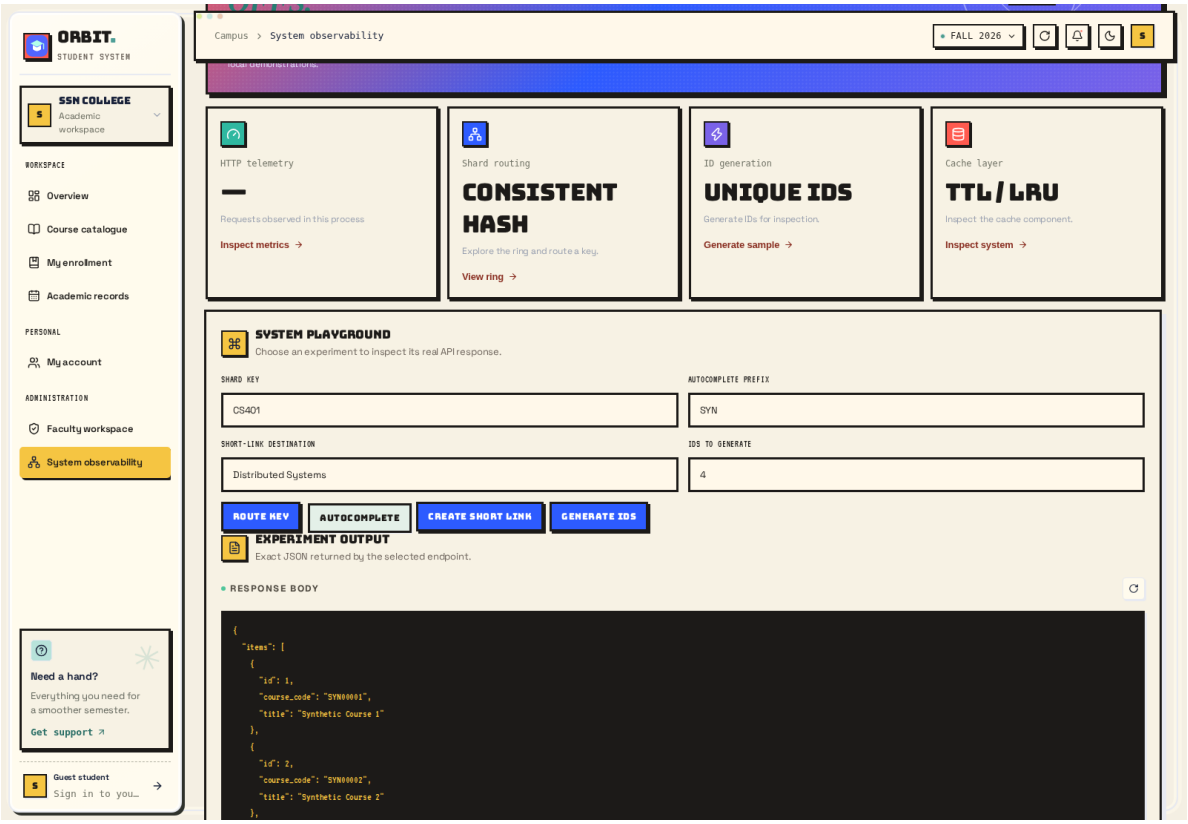


Figure 5.5. System observability workspace showing local component experiments.

Chapter 6

Data integrity and domain model

6.1 Separate catalogue identity from a term-specific offering

A course and an offering represent different concepts. A course describes reusable catalogue metadata (for example, a course code and title). An offering is a particular instance in a term or section with a status and a finite capacity. Enrollments should reference the offering, not merely the course, because students enroll in a scheduled instance. This distinction permits the same course to run in multiple terms without conflating seat counts or enrollment rosters.

6.2 Integrity rules and enforcement layers

The API validates request shape and access scope; PostgreSQL must enforce invariants that must survive concurrent requests and alternate call paths. Application validation improves error messages but cannot replace a database constraint. The key invariant for an offering is

$$0 \leq enrolled_count \leq capacity. \tag{6.1}$$

A unique student/offering relationship prevents duplicate active enrollment rows. The conditional update narrows the capacity decision to one database operation. The insert and counter update share a transaction so a uniqueness failure cannot leave the counter incremented without a matching enrollment.

Table 6.1. Integrity concern, enforcement point, and failure prevented

Concern	Enforcement	Failure prevented
Offering capacity	Conditional update in the transaction	Two concurrent requests consuming the same final seat.
Duplicate enrollment	Unique database constraint plus transaction rollback	Multiple active rows for the same student/offering.
Role and ownership	API authorization checks	Student reading or modifying another student's restricted records.
Schema evolution	Versioned Alembic migrations and fresh-database check	A new installation differing silently from the intended schema.

6.3 Idempotency is not the same as uniqueness

A uniqueness constraint protects stored state, while an idempotency key helps a client safely retry a request after a timeout or lost response. These solve related but different problems. A retry may arrive after the first transaction committed even though the client did not receive the response. A production implementation should persist the key, scope it to the caller and operation, define key retention, and return the original result for a repeated request. The recorded duplicate-race test verifies the implemented endpoint's stated behavior; it does not establish a complete cross-endpoint idempotency protocol.

Chapter 7

Architecture decisions and alternatives

7.1 Decision: a modular API with one authoritative relational store

For a coursework system with tightly related academic records, a modular monolith keeps enrollment, authorization, and database transaction boundaries in one deployable service. This avoids introducing network failure modes between separate services before the domain boundaries or operational needs justify them. PostgreSQL provides durable relational state, constraints, and transactions for relationships that must remain consistent.

7.2 Rejected or deferred alternatives

Table 7.1. Architecture alternatives and reasons for deferral

Alternative	Potential benefit	Reason deferred / trigger to revisit
Microservices for each academic domain	Independent deployment and scaling	Adds service discovery, network failures, distributed tracing, and cross-service consistency. Revisit only with clear ownership and independent scaling needs.
Cache as authority for seats	Low-latency reads	Cache state can be stale and does not provide the required transactional invariant. Keep the database authoritative.
Application-only capacity check	Simple endpoint code	Check-then-write races under concurrent requests. Use a conditional write and transaction.
Distributed cache and rate-limit service	Shared state across API workers	**Implemented** after the first edition: Redis holds the token buckets, the autocomplete cache and the ID worker slots. Revisit with Redis Cluster or a replicated deployment if availability, not sharing, becomes the requirement.
Database sharding	Horizontal data distribution and observable chunk placement	**Implemented** after the first edition: a real two-shard MongoDB cluster with `mongos`, hashed shard key, `moveChunk` and rebalance. Academic records remain in PostgreSQL; no measured PostgreSQL bottleneck justified sharding them.

7.3 Why the consistency boundary matters

The primary failure to prevent is not a slow response; it is a false academic state, such as two students being assigned one seat or a counter claiming a seat is occupied when no enrollment exists. The design therefore prioritizes transactional correctness over using an eventually consistent cache for enrollment writes. Read optimization can be introduced around that boundary only if invalidation and stale-read behavior are explicitly defined.

7.4 Operational boundaries of auxiliary components

The rate limiter, cache and shard router are backed by services that outlive the API process, so separate workers share one allowance, one cache and one placement authority rather than diverging. Their availability boundaries remain: Redis and the MongoDB members are single nodes on one host, so a restart of the API is survivable but a failure of the host is not. The hash ring demonstrates key-to-node assignment; it does not move database rows, replicate data, rebalance storage, or recover failed nodes — `moveChunk` is the operation that moves data. The crawler is additionally an outbound-network boundary and must be reviewed for private-address access, DNS rebinding, redirects, request budgets, and response-size limits before external use.

Chapter 8

Test strategy, reproducibility, and evidence quality

8.1 Layered verification strategy

The checks are deliberately layered because no single green command proves overall correctness:

1. **Static checks:** Ruff catches selected Python style and correctness issues; the frontend production build checks TypeScript and bundling. Neither executes all runtime paths.
2. **Database migration checks:** applying all revisions to a fresh database tests bootstrap reproducibility; a schema-drift check tests whether the current models imply unrecorded migration operations.
3. **Automated tests:** the recorded suite exercises API and domain assertions against PostgreSQL rather than relying only on an in-memory substitute.
4. **Targeted concurrency tests:** simultaneous requests test the two named enrollment races and assert both response outcomes and final stored state.
5. **Smoke checks:** live endpoints verify a small set of routes can respond; this is not broad endpoint coverage.
6. **Browser checks:** navigation and screenshots confirm the tested UI paths rendered without observed JavaScript errors in that run. They do not prove accessibility or all-device compatibility.
7. **Load experiments:** Locust observations characterize a short offered workload and should be repeated with controlled conditions before drawing capacity conclusions.

8.2 Isolation and repeatability

The recorded run used a dedicated PostgreSQL 17.11 container and separate databases for synthetic demonstration data and the test suite, on port 55432. This separation was intentional: the existing local database was not the target of the final verification. A repeatable test record should include the commit or source snapshot, dependency lock state, database version, migration revision, test command, environment flags, seed-data procedure, and raw logs. The values in this report describe the recorded run and should not be assumed to describe a later code snapshot unless the checks are repeated.

8.3 What the evidence does and does not establish

8.4 Benchmark interpretation

The reported request rate is a property of a specific test setup, request mix, duration, local machine, and database state. It is not a capacity limit. The 20-second Locust window is short

Table 8.1. Evidence strength and limits

Evidence	Supports	Does not establish
23 passing automated tests	Assertions in the recorded test suite pass on the configured PostgreSQL database.	Absence of untested defects, security vulnerabilities, or all possible race conditions.
20-request/five-seat race	Capacity invariant held for the tested concurrent workload.	Behavior under arbitrary load, multiple regions, or database failover.
Fresh migrations and schema check	Schema can be created from the recorded migration history without detected drift.	Disaster recovery, rollback safety for every future migration, or production data migration.
Five live smoke checks	Five named endpoints returned HTTP 200.	Correct authorization for every route or complete API health.
Single-run Locust sample	Observed latency and errors for one short local workload.	Stable throughput, capacity ceiling, statistical confidence, or production SLOs.
Browser screenshots and no observed JS errors	Selected pages rendered during the capture session.	Accessibility conformance, cross-browser correctness, or complete user acceptance.

enough that startup, connection warm-up, cache state, and test scheduling can materially affect results. The cache-enabled and cache-disabled autocomplete runs also have different total request counts, so their latency difference is a useful hypothesis rather than a controlled causal estimate. The observed 35.95

A stronger follow-up benchmark should record hardware and OS, exact code revision, database configuration and dataset size, warm-up period, steady-state duration, user arrival pattern, per-endpoint request mix, errors/timeouts, and CPU/memory for API, database, and load generator. Repeat at least several runs per configuration; report median and spread rather than only one point estimate. Test realistic and overload scenarios separately, and do not tune a limiter solely to maximize requests per second.

Chapter 9

Security, privacy, and release readiness

9.1 Current safeguards and residual risks

The prototype includes role/self-access checks for selected operations and uses synthetic data in the recorded evidence. Staff promotion is a trusted local command-line action. Token refresh/revocation and production identity integration are not implemented. A local demonstration should not be exposed to the public internet or populated with real student records on the strength of the current tests.

The crawler deserves separate threat analysis because a user-controlled URL can make the server initiate network requests. Before enabling it in a reachable deployment, enforce an allowlist or strict public-address policy, resolve and validate addresses at connection time, re-check redirects, reject loopback/private/link-local/reserved IP ranges, cap response size and time, and log/limit requests. DNS validation alone can be insufficient if the resolved address changes between validation and connection.

9.2 Release gates

Table 9.1. Minimum next gates before a non-local deployment

Gate	Required work	Evidence to retain
Identity and authorization	Production identity integration, token lifecycle, role policy review, negative authorization tests.	Threat model and automated allow/deny matrix.
Data protection	Secrets management, TLS termination, retention policy, backups, restore drills, audit of sensitive access.	Configuration review and successful restore record.
Distributed operation	Shared limiter/cache where required, connection pool limits, health checks, graceful shutdown.	Multi-worker integration tests and failure tests.
Performance and reliability	Repeated load tests, saturation tests, DB connection/lock observations, explicit SLO targets.	Reproducible benchmark report with raw results.
Outbound crawler	SSRF/DNS-pinning controls and network-enabled security tests.	Test matrix covering private IPs, redirects, DNS changes, and size/time limits.
Browser correctness	Accessibility audit, cross-browser and responsive regression.	Automated accessibility findings and viewport/browser matrix.

Chapter 10

Performance observations and limitations

10.1 Postman and Apache JMeter verification — 21 October 2026

Postman Desktop 12.33.2 was installed, and the exported collection was executed with Newman 6.2.3; Apache JMeter 5.6.3 was used for the controlled and overload runs. The API ran locally at `127.0.0.1:8000` against the existing development database and synthetic records. The Postman collection creates one uniquely named synthetic student account; no real student data were used. The retained machine-readable run summary has response bodies and bearer tokens redacted.

Table 10.1. Tool-assisted API verification, 21 October 2026

Tool / scenario	Observed result	Interpretation
Postman collection via Newman (fresh rerun)	13 requests; 24/24 assertions passed; average 20 ms	Checked health, course/offering reads, autocomplete/cache state, 25 unique IDs, ring/metrics metadata, missing authentication, registration/login, student role denial, and invalid credentials.
Apache JMeter controlled read workload (fresh rerun)	10 threads; 2 s ramp; 20 s duration; 185 samples; 0 errors; mean 4.95 ms; median 4 ms; p95 10 ms; 9.84 req/s	A paced local read-only baseline across seven endpoints. This is one short run on a shared laptop, not a production capacity estimate.
Apache JMeter unpaced overload experiment (fresh rerun)	48,459 samples; 40,372 HTTP 429; 8,081 HTTP 200; 6 socket timeouts	Deliberately unpaced traffic; 83.32% of samples failed, mostly expected HTTP 429 admission-control responses. Six transport timeouts also occurred, so this is not a clean all-429 result or a baseline.
Resource-sampled paced repeat	185 samples; 0 errors; mean 7.37 ms; p95 20 ms; 9.87 req/s	Separate run sampled with pidstat and podman stats: API CPU mean 2.65%, peak 5.0%, RSS 14–30 MiB; PostgreSQL container memory 11–22 MB. Not host-wide telemetry.
PostgreSQL automated suite	23 passed	Rerun against a dedicated PostgreSQL test database after the ID serialization fix, including both concurrent enrollment invariants.
Ruff static checks	Passed	Application, tests, scripts, and migrations.

The JMeter plan is read-only and does not prove database transaction correctness; the PostgreSQL concurrency tests establish the specific enrollment invariants. The rate limiter's counters now live in Redis and are shared by every worker, so multi-worker quota behaviour is represented in the stack verification below rather than absent. The first ID uniqueness assertion exposed JavaScript precision loss when 64-bit IDs were emitted as JSON numbers. The API now emits IDs as strings and the regression test asserts the string contract. This fixes the API representation issue; worker slots are now claimed in Redis, but the generator is still not a clock-synchronised distributed ID

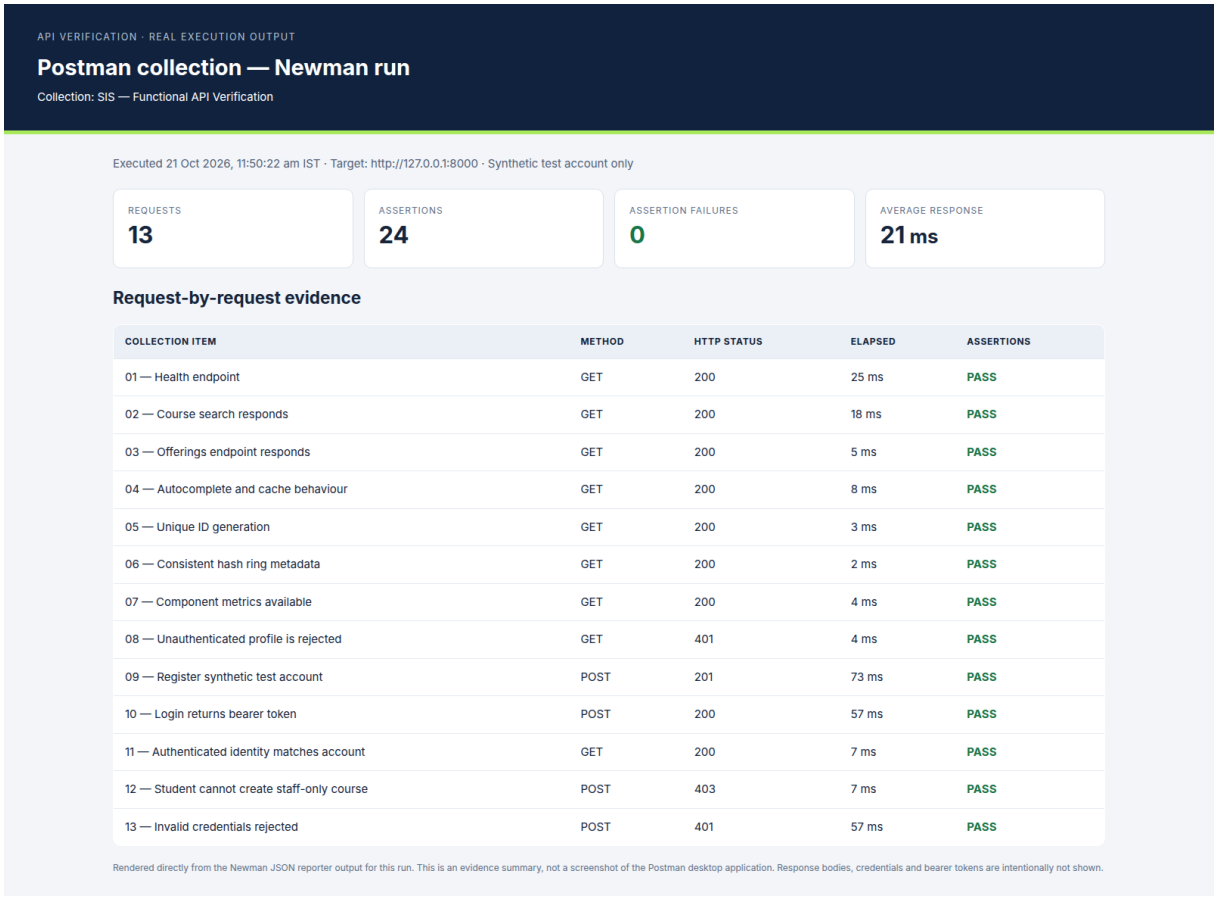


Figure 10.1. Evidence summary rendered from the fresh Newman JSON output for the Postman collection. Expected 401 and 403 responses pass their negative-security assertions; this image is not the Postman desktop UI.

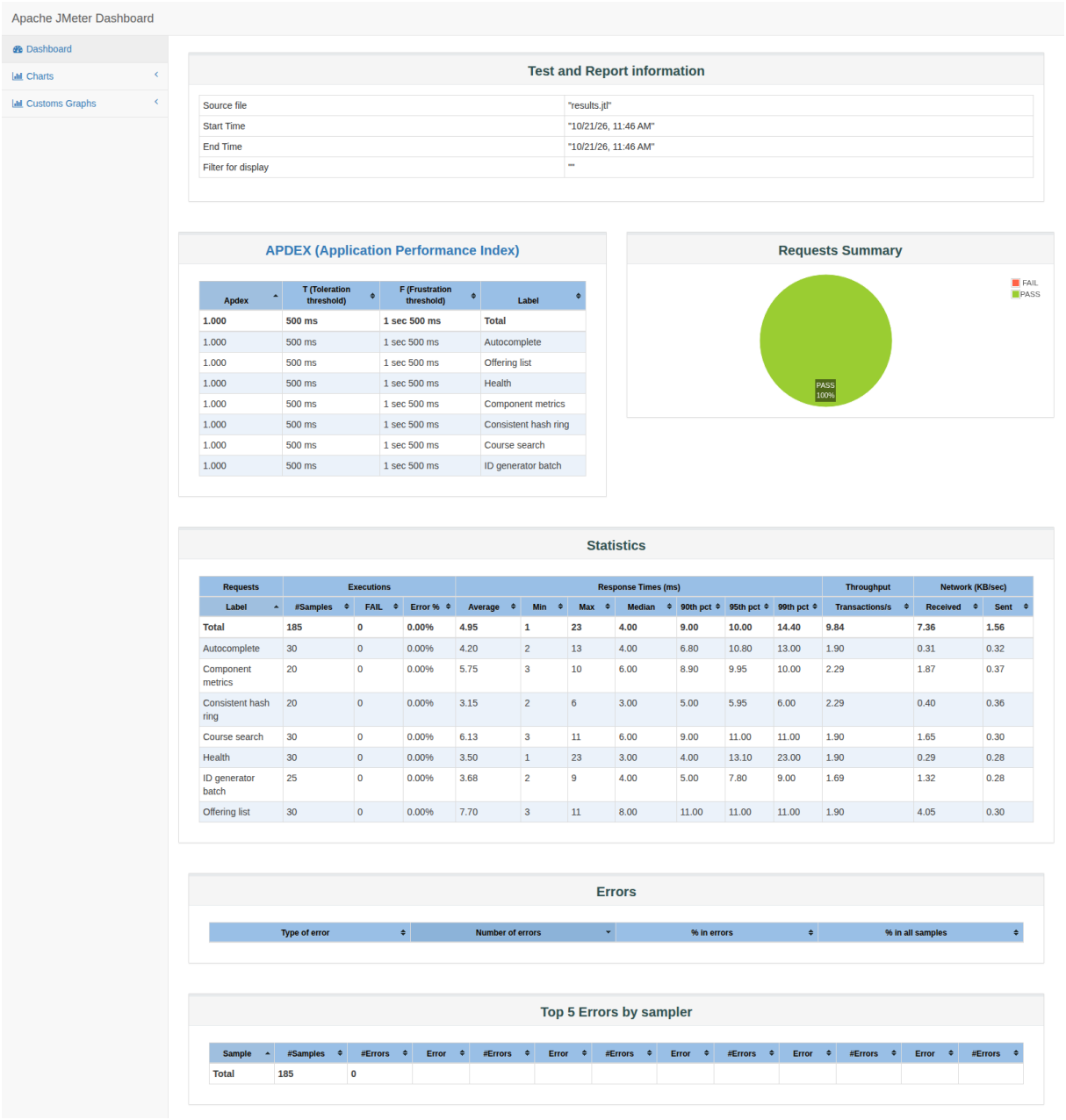


Figure 10.2. Apache JMeter HTML dashboard for the paced read-only baseline: 185 samples and zero failed requests.

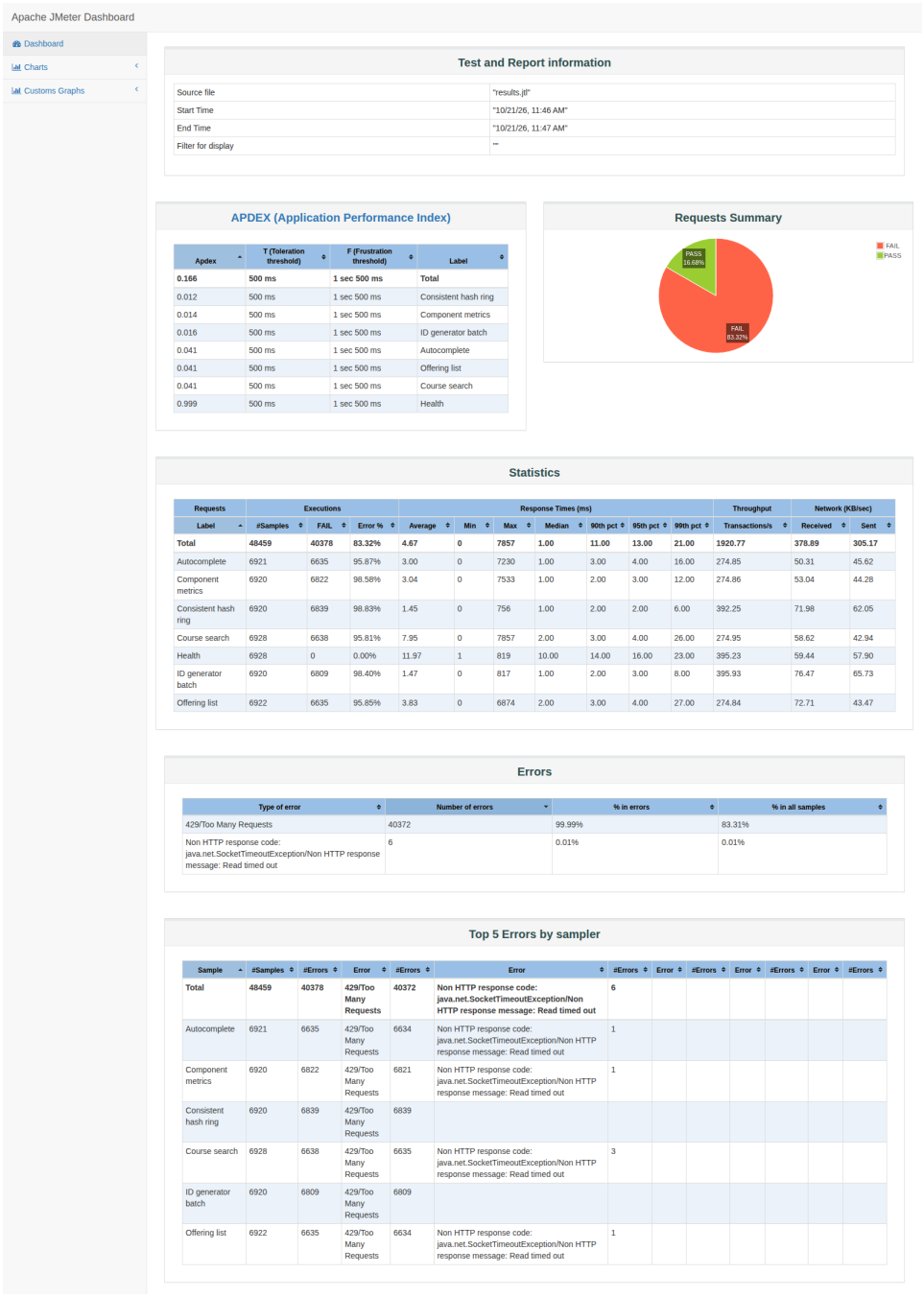


Figure 10.3. Unpaced overload produced HTTP 429 responses from the token-bucket limiter. The fresh rerun had 83.32% failed samples, including six socket timeouts as well as expected HTTP 429 responses. This overload result must not be combined with baseline results.

service.

10.2 Historical local measurements

The 19 October Locust runs used 10 users, a spawn rate of two users per second, and a 20-second limit. These are single-run local observations, not fresh measurements from the 20 October verification and not production capacity claims.

Table 10.2. Historical single-run Locust observations (19 October 2026)

Workload	Requests	req/s	p95 ms	Failures
midrule Course search/list/offering mix	681	35.82	10	0
Authenticated profile and academic reads	669	35.27	12	0
Student account registration writes	370	19.47	83	0
Autocomplete, cache enabled (pre-Redis)	675	35.55	5	0
Autocomplete, cache disabled (pre-Redis)	659	34.68	10	0
Course-search overload, limiter enabled	676	35.61	10	243 HTTP 429

The cache-enabled autocomplete run reported mean latency of 2.88 ms versus 5.85 ms without caching, roughly 50.8% lower in this run. Median latency fell from 5 ms to 2 ms and p95 from 10 ms to 5 ms. This is a short, single-run observation. Repeated runs with controlled conditions are required before generalizing the effect.

The overload run returned 243 HTTP 429 responses out of 676 requests (35.95%). This demonstrates admission control under the tested offered load, but also indicates that the quota needs tuning against legitimate traffic and service latency. A separate sample observed approximately 7.4% average CPU of one logical CPU and 90,660 KiB RSS for the API process; this was not total host or database utilization.

10.3 Stack verification after the infrastructure migration

A later run replaced every in-process store with a real service and measured the result. `scripts/verify_stack.py` executes each claim against the live containers and writes the raw output to `stack_verification.json`.

The automated suite reports 37 passed and 2 skipped. Both skips are deliberate: the PostgreSQL concurrency race requires an explicitly provided `DATABASE_URL`, and the live crawl is opt-in so the suite never contacts a third-party site by default.

10.4 Limitations

- No production deployment, availability, security, or capacity certification was performed.
- Redis and the MongoDB members are single nodes on one host: shared state is verified, failover is not.
- The consistent-hash ring is a startup snapshot of the shard set and moves no data; `mongos` owns placement.
- The crawler now works against the live network but remains same-host and serial; DNS rebinding is not defended against.

Table 10.3. Measured behaviour against Redis 7 and a two-shard MongoDB 7 cluster (21 October 2026)

Check	Observed	What it establishes
midrule Rate limiter across processes	capacity 3 spent by worker A; worker B refused	One allowance is shared, not duplicated per process
Cache TTL	TTL 1 s, then `None`	Expiry is server-side, not a Python dict
ID worker slots	slots 1 and 2; 1,000 IDs, 1,000 unique	Concurrent processes cannot claim the same slot
Shard distribution	249 / 251 documents, 3 chunks each	`mongos` places records across both shards
`moveChunk`	129 / 371, all 500 keys readable	Chunk migration moves real data without loss
Rebalance	256 / 244, 3 chunks each	The balancer restores an even split
Live crawl	`www.iana.org`: robots parsed, 5 pages stored	The network path works against a real host
Crawl resume	5 URLs taken from the persisted frontier, 0 refetched	The frontier is durable and deduplication is real
bottomrule		

- The cache-on/cache-off Locust rows predate the Redis migration and were not re-measured; treat them as historical comparison only.
- Historical Locust results are short single runs and lack repeated confidence intervals.
- The API now serializes 64-bit identifiers as JSON strings to preserve exactness in JavaScript clients; older clients expecting numeric IDs must be updated.

Chapter 11

Mobile layout quality assurance

Initial visual QA at a 390 × 844 viewport found that the main content retained its desktop left margin after the sidebar changed into a mobile overlay. A scoped CSS rule under the 760 px breakpoint reset the content margin to zero and constrained the main area to the viewport. The change did not alter component markup, typography, colors, glass surfaces, or desktop layout rules.

The recorded Playwright checks covered viewport widths of 320, 390, 430, 768, 1024, and 1440 px. The mobile drawer was opened and dismissed at 320, 390, 430, and 700 px. The document width matched the viewport and no browser JavaScript errors were observed during the run.

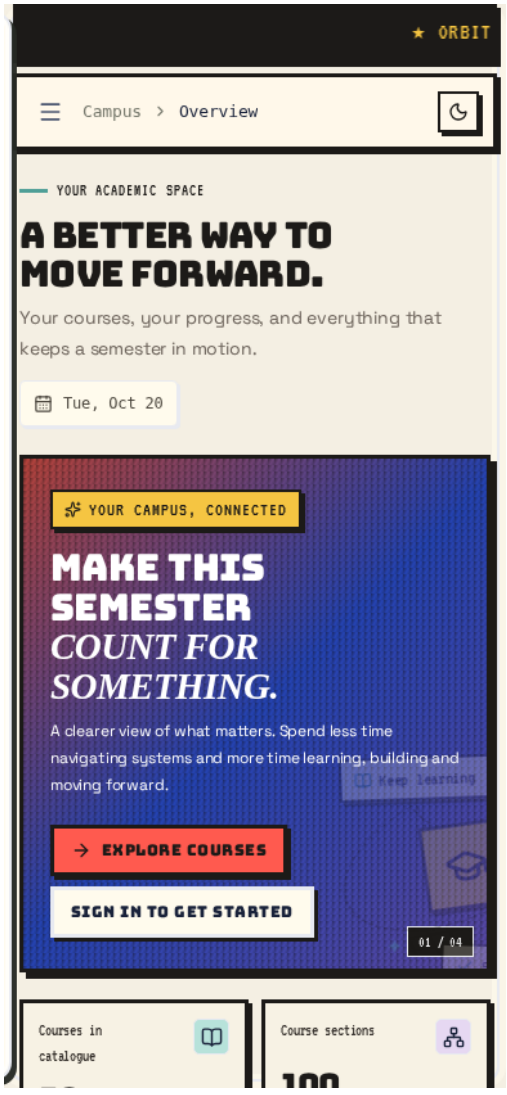


Figure 11.1. Mobile layout after the responsive offset fix, captured at 390 × 844. This is targeted geometry regression evidence, not a full accessibility audit.

Chapter 12

Conclusion and next steps

The strongest verified design property is the enrollment consistency boundary: the seat reservation and enrollment insert share a PostgreSQL transaction, and targeted concurrency tests passed in the isolated verification environment. The broader local verification also recorded a passing test suite, fresh migrations, no schema drift, a successful frontend build, API smoke checks, and stable browser navigation.

Before treating the prototype as a deployable service, the following work remains:

1. Add token revocation/refresh and a production identity provider; expand authentication and rate-limit abuse tests.
2. Replicate Redis and the MongoDB members, and add failure-recovery exercises, before making availability claims.
3. Repeat load tests over longer durations and record API, database, and load-generator resource time series; re-measure the Redis cache against the historical numbers.
4. Add backup/restore drills before claiming operational readiness.
5. Review crawler DNS-pinning and SSRF protections before exposing it beyond coursework.
6. Add contract tests to ensure all API consumers preserve ID strings and do not coerce them to JavaScript Number.

The results establish a verified local baseline. They do not establish production readiness. That distinction should remain explicit in demonstrations, submissions, and future benchmark reports. The report structure, figure placement, and accessible table conventions follow established technical-report guidance [1, 2].

References

[1] Carnegie Mellon University Robotics Institute. *Technical Report Format*. Guidance on technical-report structure, readable type, figures, tables, and citations. https://www.ri.cmu.edu/technical-report-format/.

[2] National Institute of Standards and Technology. *NIST Technical Series Publications Author Instructions*. Guidance on report organization, plain language, accessible tables, figure captions, and references. https://www.nist.gov/nist-research-library/nist-technical-series-publications-author-instructions.

[3] PostgreSQL Global Development Group. *Transaction Isolation*. https://www.postgresql.org/docs/16/transaction-iso.html.

[4] PostgreSQL Global Development Group. *Explicit Locking*. https://www.postgresql.org/docs/17/explicit-locking.html.

[5] Locust. *What is Locust?*. Load-testing framework documentation. https://docs.locust.io/en/2.20.2/what-is-locust.html.

[6] Carnegie Mellon University Libraries. *How to Make a Science Research Poster: Design*. https://guides.library.cmu.edu/scienceposters/design.

[7] Nielsen Norman Group. *Visual Hierarchy in UX*. https://www.nngroup.com/articles/visual-hierarchy-ux-definition/.

Chapter A

Reproduction checklist

Run these commands from the `sis-app` directory with the isolated verification database available. Substitute the locally configured test password; do not commit credentials.

```
export DATABASE_URL='postgresql+psycopg://sis:<local-test-password>@127.0.0.1:55432/sis_verify'
uv run alembic upgrade head
uv run alembic check
SIS_RATE_LIMIT_ENABLED=false uv run pytest -q
uv run ruff check app tests scripts migrations
cd frontend && npm run build
```

Chapter B

Evidence interpretation

- **Automated tests.** Evidence for the assertions exercised by the suite against the recorded database configuration.
- **Concurrency tests.** Evidence for the tested capacity and duplicate-enrollment races, not a universal race-freedom proof.
- **UI screenshots.** Evidence that the displayed workflows were captured from the local frontend with synthetic data, not proof of production deployment.
- **Historical load tests.** Short local measurements that require repetition before supporting capacity estimates.
- **Process-local components.** Functional demonstrations only, not distributed infrastructure.